

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I __B. Kishan ram/192421268_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Kishan Ram

Q1. SQL queries using joins and subqueries (BL3–BL4)

(a) Query using a JOIN

Problem: List every student's name together with the title of each course they are enrolled in and the grade obtained.

```
SELECT S.Name, C.CTitle, E.Grade
FROM   Student S
JOIN   Enrollment E ON S.StudentID = E.StudentID
JOIN   Course C      ON E.CourseID = C.CourseID
ORDER BY S.Name;
```

Logic: The three tables are related through foreign keys. Student is joined to Enrollment on StudentID to attach each enrollment record to the correct student, and Enrollment is joined to Course on CourseID to replace the course code with its title. The JOIN evaluates matching rows first, so only students who actually have an enrollment row appear.

Expected Output

Name	CTitle	Grade
Arjun	DBMS	A
Arjun	Operating Systems	B
Divya	Computer Networks	A
Kiran	DBMS	A
Kiran	Operating Systems	A
Meena	Web Technology	B
Rahul	DBMS	C

(b) Query using a Subquery

Problem: List the names of students who are enrolled in the 'DBMS' course.

```
SELECT Name
FROM   Student
WHERE  StudentID IN (SELECT StudentID
                     FROM   Enrollment
                     WHERE  CourseID = (SELECT CourseID
                                       FROM   Course
                                       WHERE  CTitle = 'DBMS'));
```

Logic: The innermost subquery resolves the course title 'DBMS' to its CourseID (C201). The middle subquery then returns the set of StudentIDs enrolled in that course. The outer query filters Student rows whose StudentID lies in that set. Subqueries are useful here because the filtering condition (a specific course) is naturally expressed as a nested lookup rather than a join, and no columns from Course or Enrollment need to appear in the final result.

Expected Output: Arjun, Kiran, Rahul

Q2. Relational Algebra for multi-condition queries (BL4–BL5)

Problem: Retrieve the names of CSE-department students who scored grade 'A' in course C201.

Relational Algebra Expression

```

$$\pi \text{ Name } ( \sigma (\text{Dept} = \text{'CSE'}) (\text{Student}) \bowtie (\text{Student.StudentID} = \text{Enrollment.StudentID}) \sigma (\text{CourseID} = \text{'C201'} \wedge \text{Grade} = \text{'A'}) (\text{Enrollment}) )$$

```

Step-by-step:

- $\sigma(\text{Dept} = \text{'CSE'}) (\text{Student})$ — selects only CSE-department student rows.
- $\sigma(\text{CourseID} = \text{'C201'} \wedge \text{Grade} = \text{'A'}) (\text{Enrollment})$ — selects enrollment rows satisfying both conditions simultaneously (a compound predicate using $\wedge$).
- The two intermediate relations are combined with a natural join ($\bowtie$) on StudentID, keeping only students who appear in both.
- $\pi \text{ Name}$ — projects only the Name attribute for the final result.

Equivalent SQL

```
SELECT S.Name
FROM   Student S JOIN Enrollment E ON S.StudentID = E.StudentID
WHERE  S.Dept = 'CSE' AND E.CourseID = 'C201' AND E.Grade = 'A';
```

Result on the sample data: Arjun, Kiran (both CSE students with grade A in C201).

Q3. Real-world schema design — Banking System (BL6)

Scenario: A bank wants to track customers, their accounts, and the transactions made on those accounts.

Relational Schema

```
Customer      (CustID, CName, City, Phone)
Account       (AccNo, CustID, AccType, Balance, BranchName)
Transaction   (TxnID, AccNo, TxnType, Amount, TxnDate)

Primary Keys : Customer.CustID, Account.AccNo, Transaction.TxnID
Foreign Keys : Account.CustID      -> Customer.CustID
               Transaction.AccNo   -> Account.AccNo
```

Sample Data

CustID	CName	City	Phone
CU1	Ramesh	Chennai	98400xxxxx
CU2	Sneha	Madurai	97900xxxxx

AccNo	CustID	AccType	Balance	BranchName
A001	CU1	Savings	50000	Anna Nagar
A002	CU1	Current	120000	Anna Nagar
A003	CU2	Savings	30000	Madurai Main

SQL — Data Retrieval

List all savings accounts with balance above ₹40,000, along with the customer name.

```
SELECT C.CName, A.AccNo, A.Balance
FROM   Customer C JOIN Account A ON C.CustID = A.CustID
WHERE  A.AccType = 'Savings' AND A.Balance > 40000;
```

SQL — Data Update

Credit ₹5,000 to account A001 (a deposit transaction) and update its balance.

```
UPDATE Account
SET    Balance = Balance + 5000
WHERE  AccNo = 'A001';

INSERT INTO Transaction (TxnID, AccNo, TxnType, Amount, TxnDate)
VALUES ('T101', 'A001', 'Deposit', 5000, '2026-08-01');
```

The two statements are normally wrapped in a single transaction (BEGIN...COMMIT) so that the balance update and the transaction log entry either both succeed or both fail, preserving consistency.

Q4. Nested subquery — predict output & execution steps (BL4–BL6)

Given query (run against the Student–Course–Enrollment database):

```
SELECT Name
FROM   Student
WHERE  StudentID IN (
        SELECT StudentID
        FROM   Enrollment
        WHERE  CourseID IN (
                SELECT CourseID
                FROM   Course
            )
    )
```

```
WHERE Dept = 'CSE'
)
);
```

Execution, step by step

- Step 1 (innermost query): `SELECT CourseID FROM Course WHERE Dept='CSE'` evaluates first and returns {C201, C203} — the CSE-owned courses.
- Step 2 (middle query): `SELECT StudentID FROM Enrollment WHERE CourseID IN {C201, C203}` scans Enrollment and returns every StudentID enrolled in either course: {S101, S103, S105, S101, S103} → distinct set {S101, S103, S105}.
- Step 3 (outer query): `SELECT Name FROM Student WHERE StudentID IN {S101, S103, S105}` looks up matching students in Student.

SQL evaluates nested (uncorrelated) subqueries from the innermost outward, exactly once each, because they don't reference the outer query's row — the result is reused as a constant set for the outer WHERE clause.

Predicted Output: Arjun, Kiran, Rahul

Q5. Joins vs Subqueries — comparison and evaluation (BL5)

Problem: Find the names of students who are enrolled in course 'C201'.

Approach 1 — JOIN

```
SELECT DISTINCT S.Name
FROM   Student S JOIN Enrollment E ON S.StudentID = E.StudentID
WHERE  E.CourseID = 'C201';
```

Approach 2 — Subquery

```
SELECT Name
FROM   Student
WHERE  StudentID IN (SELECT StudentID FROM Enrollment WHERE CourseID = 'C201');
```

Evaluation

Aspect	JOIN	Subquery
Result	Arjun, Kiran, Rahul	Arjun, Kiran, Rahul (same)
Readability	Clear when columns from both tables are needed	Clearer when only a membership check is needed
Optimizer behaviour	Most RDBMS optimizers convert IN-subqueries into semi-joins internally, so performance often converges	Correlated subqueries (re-executed per outer row) can be slower on large tables without proper indexing

Aspect	JOIN	Subquery
Index use	Benefits directly from an index on the join column (Enrollment.StudentID)	Benefits from an index on the filtering column (Enrollment.CourseID)
Extensibility	Easy to add extra columns (e.g., Grade) without restructuring	Adding extra columns forces conversion to a join anyway

Conclusion: for this particular query, both are functionally and, on a modern optimizer, roughly equally efficient because a single equality condition on an indexed foreign key is involved. Joins scale better as more attributes from multiple tables are needed in the output; subqueries read more naturally for pure existence/membership checks and avoid duplicate rows without needing DISTINCT.

Q6. Views for restricted access — design and justification (BL5–BL6)

Scenario: The registrar's office should see only enrollment/grade information, without exposing sensitive student attributes such as City or Age, and a course coordinator should see only enrollments for the courses their department teaches.

View 1 — Restricting Columns

```
CREATE VIEW StudentGradeView AS
SELECT S.StudentID, S.Name, C.CTitle, E.Grade, E.Semester
FROM   Student S
JOIN   Enrollment E ON S.StudentID = E.StudentID
JOIN   Course C      ON E.CourseID = C.CourseID;
```

View 2 — Restricting Rows (department-scoped)

```
CREATE VIEW CSEEnrollmentView AS
SELECT S.Name, C.CTitle, E.Grade
FROM   Student S
JOIN   Enrollment E ON S.StudentID = E.StudentID
JOIN   Course C      ON E.CourseID = C.CourseID
WHERE  C.Dept = 'CSE';
```

Usage

```
SELECT * FROM StudentGradeView WHERE Grade = 'A';
```

Justification

- Security: Age and City are never exposed to a user who is only granted SELECT on the view, since the view's column list simply omits them — this is column-level access control without needing per-column GRANT statements on the base table.
- Row-level security: CSEEnrollmentView hides rows from other departments, letting a CSE coordinator's account be granted access only to that view.

- Abstraction: The view hides the three-table join behind a single virtual table, so downstream users/reports write simple single-table SELECTs and are insulated from later schema changes (e.g., splitting Course into Course and CourseOffering) as long as the view definition is updated accordingly.

Q7. Relational Algebra $\rightleftharpoons$ SQL — representation differences (BL4)

Problem A: Names of students who are NOT enrolled in any course.

Relational Algebra

```
 $\pi \text{ Name } ( \pi \text{ StudentID}(\text{Student}) - \pi \text{ StudentID}(\text{Enrollment}) ) \bowtie \text{Student}$ 
(i.e., set-difference on StudentID, then join back to Student to fetch Name)
```

Equivalent SQL

```
SELECT Name FROM Student
WHERE StudentID NOT IN (SELECT StudentID FROM Enrollment);
```

On the sample data both return an empty set, since every student has at least one enrollment.

Problem B: Titles of courses taught by faculty in the 'CSE' department.

Relational Algebra

```
 $\pi \text{ CTitle } ( \text{Course} \bowtie (\text{Teaches} \bowtie \sigma (\text{Dept} = \text{'CSE'}) (\text{Faculty})) )$ 
```

Equivalent SQL

```
SELECT DISTINCT C.CTitle
FROM   Course C
JOIN   Teaches T   ON C.CourseID = T.CourseID
JOIN   Faculty F   ON T.FacultyID = F.FacultyID
WHERE  F.Dept = 'CSE';
```

Result: DBMS, Operating Systems.

Representation differences

- Relational algebra is a procedural, operator-based notation (σ , π , $\bowtie$, $-$, $\cup$, $\cap$); it specifies an order of operations explicitly and each operator produces an intermediate relation.
- SQL is declarative: it states what is wanted (SELECT/FROM/WHERE), and the query optimizer chooses an actual execution plan, which may reorder the algebraic operations for efficiency (e.g., pushing selections before joins).
- Set difference ($-$) has a direct SQL analogue (NOT IN / EXCEPT / NOT EXISTS), while natural join ($\bowtie$) most commonly maps to an explicit JOIN...ON with an equality condition.

Q8. Optimizing inefficient SQL queries (BL5)

Inefficient query: For every student, print the name and how many courses they are enrolled in, using a correlated subquery in the SELECT list (executed once per row of Student).

```
-- INEFFICIENT (correlated subquery re-run for every student row)
SELECT S.Name,
       (SELECT COUNT(*) FROM Enrollment E WHERE E.StudentID = S.StudentID) AS
NumCourses
FROM   Student S;
```

Restructured using a JOIN + GROUP BY

```
-- OPTIMIZED
SELECT S.Name, COUNT(E.CourseID) AS NumCourses
FROM   Student S
LEFT JOIN Enrollment E ON S.StudentID = E.StudentID
GROUP BY S.StudentID, S.Name;
```

Further optimization — indexing

```
CREATE INDEX idx_enroll_studentid ON Enrollment(StudentID);
```

Further optimization — a reusable view

```
CREATE VIEW StudentCourseCount AS
SELECT S.StudentID, S.Name, COUNT(E.CourseID) AS NumCourses
FROM   Student S LEFT JOIN Enrollment E ON S.StudentID = E.StudentID
GROUP BY S.StudentID, S.Name;
```

Evaluation of the improvement

- The correlated subquery has $O(n \times m)$ behaviour in the worst case (n students, m enrollments scanned per student, absent an index). The single LEFT JOIN + GROUP BY scans each table once, giving the optimizer a single execution plan to tune, typically $O(n + m)$ with a hash or merge join.
- An index on Enrollment.StudentID lets the join use an index seek instead of a full scan, which matters most as Enrollment grows.
- Using LEFT JOIN (instead of INNER JOIN) preserves students with zero enrollments, showing NumCourses = 0, which the correlated subquery version also does correctly — an important correctness detail to keep during optimization.
- Wrapping the pattern in a view avoids re-writing the join every time the count is needed, and lets the DBA optimize the underlying query once.

Q9. Grouping, aggregation, and nested queries (BL6)

Problem: For each course, show the course title and the number of students who scored grade 'A', but only for courses where more than one student scored an 'A' — and additionally list only courses whose enrollment count is above the average enrollment per course.

```
SELECT C.CTitle,
       COUNT(*) AS TopScorers
FROM   Course C
JOIN   Enrollment E ON C.CourseID = E.CourseID
WHERE  E.Grade = 'A'
AND    C.CourseID IN (
    SELECT CourseID
    FROM   Enrollment
    GROUP BY CourseID
    HAVING COUNT(*) > (
        SELECT AVG(cnt) FROM (
            SELECT COUNT(*) AS cnt
            FROM   Enrollment
            GROUP BY CourseID
        ) AS PerCourse
    )
)
GROUP BY C.CTitle
HAVING COUNT(*) > 1;
```

Explanation

- The innermost derived table PerCourse groups Enrollment by CourseID and counts rows, giving the enrollment size of every course.
- AVG(cnt) over that derived table gives the average enrollment per course across the whole system.
- The middle subquery's HAVING COUNT(*) > (that average) keeps only courses whose enrollment is above average, returning their CourseIDs.
- The outer query joins Course to Enrollment, restricts to Grade='A' rows belonging only to those above-average courses, groups by course title, and keeps only titles with more than one 'A' scorer via the outer HAVING clause.

On the sample data, C201 (3 enrollments) and C203 (2 enrollments) are above the average of 1.75, and C201 has two 'A' scorers (Arjun, Kiran), so it is the only row returned.

Expected Output

CTitle	TopScorers
DBMS	2

Q10. Choosing between views, joins, and subqueries (BL5)

Given relations Student, Course, Faculty, Teaches, and Enrollment, three related requirements are analysed below, each requiring a different construct.

Requirement 1: A permanent, frequently-reused report of student names with their enrolled course titles and grades, to be queried by many downstream reports

Chosen construct: VIEW (built on a JOIN). Reasoning: the same three-table join is needed repeatedly by different consumers; a view centralizes the logic, hides the join complexity, and can be permission-scoped for restricted access, matching the requirement of reuse and abstraction.

```
CREATE VIEW StudentEnrollmentReport AS
SELECT S.Name, C.CTitle, E.Grade
FROM   Student S JOIN Enrollment E ON S.StudentID = E.StudentID
      JOIN Course C      ON E.CourseID = C.CourseID;
```

Requirement 2: A one-off query to list students together with the course they are enrolled in, for a single report where columns from both tables are needed

Chosen construct: JOIN, used directly. Reasoning: the query needs attributes from both Student and Course simultaneously in a single result row, which is exactly what a join is designed for, and there is no reuse requirement that would justify creating a view.

```
SELECT S.Name, C.CTitle
FROM   Student S JOIN Enrollment E ON S.StudentID = E.StudentID
      JOIN Course C      ON E.CourseID = C.CourseID;
```

Requirement 3: Find faculty members who do NOT teach any CSE course, where only Faculty columns are needed in the output

Chosen construct: SUBQUERY (NOT IN / NOT EXISTS). Reasoning: this is a set-membership / non-membership check confined to a single table's output (Faculty), so a subquery expresses the intent more directly than a join, and avoids the need for DISTINCT or extra grouping that a join+anti-join pattern would otherwise require.

```
SELECT FName
FROM   Faculty
WHERE  FacultyID NOT IN (
      SELECT T.FacultyID
      FROM   Teaches T JOIN Course C ON T.CourseID = C.CourseID
      WHERE  C.Dept = 'CSE'
    );
```

General decision rule used above

- Use a VIEW when the same multi-table logic will be reused across many queries/reports or when specific users must be restricted to a subset of columns/rows.
- Use a JOIN directly when the result genuinely needs columns from more than one relation in the same row, and the query is not reused often enough to justify a view.
- Use a SUBQUERY when the requirement is a membership/non-membership or aggregate-comparison test that ultimately returns columns from only one relation.